Declaring Types and Classes

DM580: Functional Programming

Casper Bach





Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 8

Learning Objectives

- Explain the benefits and drawbacks of data types in Haskell vs. OO classes
- Explain the benefits and drawbacks of type classes in Haskell
- Declare your own data types in Haskell
- Program and pattern match against custom data types in Haskell
- Write your own type classes and instances in Haskell
- Explain the difference between type synonyms (`type`) and data type declarations (`data`) in Haskell
- Use "`deriving`" clauses on data types in Haskell to derive standard type classes



Why Data Types?

Why Data Types

1. Structured data types enforce safety invariants that rule out bugs

2. Recursive data type admit well-typed traversals via recursion

3. Separation of concerns: data types separate structure from functionality — for better and worse

Unstructured Data Types

Spot the broken invariants

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)
```

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2
```

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2

fahrenheit :: Reading → Double
fahrenheit (d, u) = d * 9 / 5 + 32
```

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2

fahrenheit :: Reading → Double
fahrenheit (d, u) = d * 9 / 5 + 32

comfortable :: Reading → Bool
comfortable (d, u) = 18 ⩽ d && d < 23
```

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2

fahrenheit :: Reading → Double
fahrenheit (d, u) = d * 9 / 5 + 32

comfortable :: Reading → Bool
comfortable (d, u) = 18 ⩽ d && d < 23
```

What if u1 and u2 differ?

Unstructured Data Types

Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2

fahrenheit :: Reading → Double
fahrenheit (d, u) = d * 9 / 5 + 32

comfortable :: Reading → Bool
comfortable (d, u) = 18 ⩽ d && d < 23
```

What if u1 and u2 differ?

What if u=1?

Unstructured Data Types

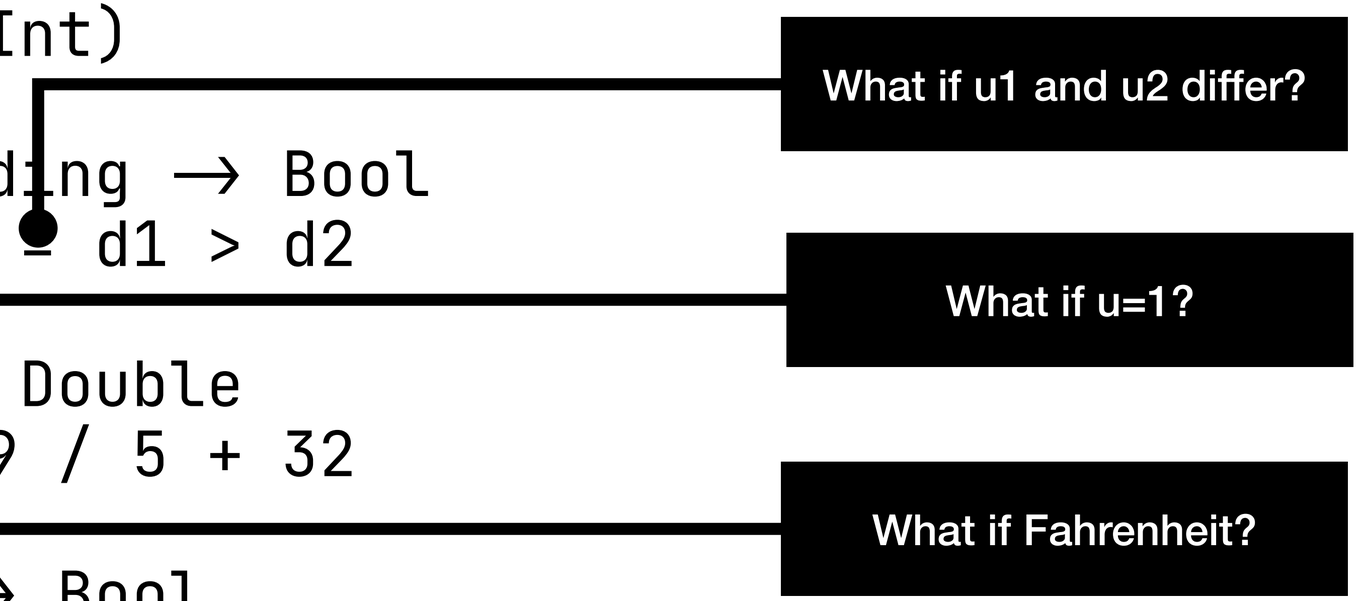
Spot the broken invariants

```haskell
-- A temperature reading: (degrees, unit)
--   unit: 0 = Celsius, 1 = Fahrenheit
type Reading = (Double, Int)

warmer :: Reading → Reading → Bool
warmer (d1, u1) (d2, u2) = d1 > d2

fahrenheit :: Reading → Double
fahrenheit (d, u) = d * 9 / 5 + 32

comfortable :: Reading → Bool
comfortable (d, u) = 18 ⩽ d && d < 23
```



Structured Data Types Enforce Invariants

Structured Data Types Enforce Invariants

```
data Temp = Celsius Double | Fahrenheit Double
```

Structured Data Types Enforce Invariants

```
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...
```

Structured Data Types Enforce Invariants

```
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9
```

Structured Data Types Enforce Invariants

```
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9

comfortable :: Temp → Bool
comfortable t = c ⩾ 18 && c < 23
  where c = celsius t
```

Structured Data Types Enforce Invariants

```haskell
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9

comfortable :: Temp → Bool
comfortable t = c ⩾ 18 && c < 23
  where c = celsius t
```

Pattern match exposes difference so we only compare C-C / F-F.

Structured Data Types Enforce Invariants

```haskell
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9

comfortable :: Temp → Bool
comfortable t = c ⩾ 18 && c < 23
  where c = celsius t
```

Pattern match exposes difference so we only compare C-C / F-F.

Pattern match exposes difference

Structured Data Types Enforce Invariants

```haskell
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9

comfortable :: Temp → Bool
comfortable t = c ⩾ 18 && c < 23
  where c = celsius t
```

Pattern match exposes difference so we only compare C-C / F-F.

Pattern match exposes difference

Pattern match exposes difference

Structured Data Types Enforce Invariants

```haskell
data Temp = Celsius Double | Fahrenheit Double

warmer :: Temp → Temp → Bool
warmer (Celsius d1) (Celsius d2) = d1 > d2
warmer (Celsius d1) (Fahrenheit d2) = d1 > (d2 - 32) / 1.8
warmer (Fahrenheit d1) (Celsius d2) = ...

fahrenheit :: Temp → Double
fahrenheit (Celsius d) = d * 9 / 5 + 32
fahrenheit (Fahrenheit d) = d

celsius :: Temp → Double
celsius (Celsius d) = d
celsius (Fahrenheit d) = (d - 32) * 5 / 9

comfortable :: Temp → Bool
comfortable t = c ⩾ 18 && c < 23
  where c = celsius t
```

Pattern match exposes difference so we only compare C-C / F-F.

Pattern match exposes difference

Pattern match exposes difference

Explicit pre-conversion to C, otherwise ill-typed

Recursive Data Types and Traversal

- A *recursive* data type is a data type that naturally nests inside itself. For example, Peano numbers, lists, trees, etc.

```
data Nat  = Z | Suc Nat
data List = Nil | Cons Int List
data Tree = Leaf Int | Node Tree Int Tree
data Nest = Atom Int | Nest [Nest]
```

- Recursive functions traverse recursive data types, in a typed manner. Cases are *equations.* Haskell checks *exhaustiveness*.

```
toInt :: Nat → Int
toInt Z       = 0
toInt (Suc n) = 1 + toInt n
```

```
height :: Tree → Int
height (Leaf _)     = 0
height (Node l x r) =
  1 + max (height l) (height r)
```

OO Classes vs. Data in Haskell

Adding new functionality vs. adding a new type

OO Classes vs. Data in Haskell

Adding new functionality vs. adding a new type

- Say we have a program with different shapes (Circle, Rect), and we need to compute their area.

OO Classes vs. Data in Haskell

Adding new functionality vs. adding a new type

- Say we have a program with different shapes (Circle, Rect), and we need to compute their area.
- And say we need to add new functionality that computes shapes' perimeter.

OO (e.g., Java/C#)	Haskell
Hard. Must edit the interface/class definition and every existing class	**Easy.** Just write a new function `perimeter :: Shape → Area` No existing code touched.

OO Classes vs. Data in Haskell

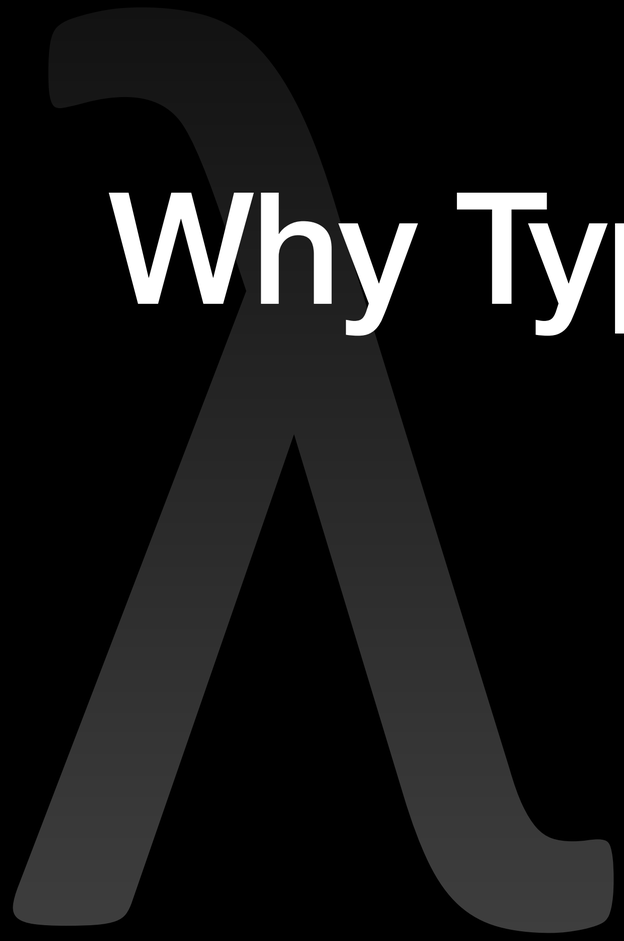
Adding new functionality vs. adding a new type

- Say we have a program with different shapes (Circle, Rect), and we need to compute their area.
- And say we need to add new functionality that computes shapes' perimeter.

OO (e.g., Java/C#)	Haskell
Hard. Must edit the interface/class definition and every existing class	**Easy.** Just write a new function `perimeter :: Shape → Area` No existing code touched.

- Now say we need to add a new shape (e.g., a Triangle)

OO (e.g., Java/C#)	Haskell
Easy. Just write a new class implementing the interface. No existing code touched.	**Hard.** Every function that consumes shapes (area, perimeter, show, etc.) must handle the new case.



Why Type Classes?

Type Classes — Similar Trade-Offs as OO Classes

Type Classes — Similar Trade-Offs as OO Classes

```
data Circle = Circ Point Double
data Rect   = Rect Point Point

class Perim a where
  perimeter :: a → Double

instance Perim Circle where perimeter = …
instance Perim Rect   where perimeter = …
```

Type Classes — Similar Trade-Offs as OO Classes

```
data Circle = Circ Point Double
data Rect   = Rect Point Point

class Perim a where
  perimeter :: a → Double

instance Perim Circle where perimeter = …
instance Perim Rect   where perimeter = …

data Tri = Tri Point Point Point

instance Perim Tri    where perimeter = …
```

Type Classes — Trade-Offs

Type Classes — Trade-Offs

- Adding a new `Perim Tri` case was **easy**. Just declare a new instance. No updates to existing code, as demonstrated on the previous slide.

Type Classes — Trade-Offs

- Adding a new `Perim Tri` case was **easy**. Just declare a new instance. No updates to existing code, as demonstrated on the previous slide.

- Adding a new function (say, `area :: a → Double`) to `Perim a` is **hard**. Requires updating all existing instances, to add the functionality.

Type Classes — Trade-Offs

- Adding a new `Perim Tri` case was **easy**. Just declare a new instance. No updates to existing code, as demonstrated on the previous slide.

- Adding a new function (say, `area :: a → Double`) to `Perim a` is **hard**. Requires updating all existing instances, to add the functionality.

- I.e., similar benefits and trade-offs as classes, but in Haskell.

Type Classes — Trade-Offs

- Adding a new `Perim Tri` case was **easy**. Just declare a new instance. No updates to existing code, as demonstrated on the previous slide.

- Adding a new function (say, `area :: a → Double`) to `Perim a` is **hard**. Requires updating all existing instances, to add the functionality.

- I.e., similar benefits and trade-offs as classes, but in Haskell.

- Type classes (or variants thereof) are in many modern languages (e.g., Rust traits, Scala implicits, Lean type classes, Agda instance arguments, etc.).

Type Classes vs. OO Classes

- OO classes use *dynamic dispatch* (i.e., which object receives a method call is only decided at run time)

- Type classes in Haskell use *static dispatch* (i.e., which type class instance we run is decided at compile time)

- Static dispatch is cheaper than dynamic: direct jump vs. indirect jump

Learning Objectives

✓ Explain the benefits and drawbacks of data types in Haskell vs. OO classes

✓ Explain the benefits and drawbacks of type classes in Haskell

- Declare your own data types in Haskell
- Program and pattern match against custom data types in Haskell
- Write your own type classes and instances in Haskell
- Explain the difference between type synonyms (`type`) and data type declarations (`data`) in Haskell
- Use "`deriving`" clauses on data types in Haskell to derive standard type classes

Declare Your Own Data Types in Haskell



Exercise 1: Data Type of Arithmetic Expressions

- Define a data type of arithmetic expressions (numbers, plus, times) in Haskell

```
data Expr = Val Int | …
```

Exercise 2: Evaluator of Arithmetic Expressions

- Define an evaluator of arithmetic expressions in Haskell

```
eval :: Expr → Int
eval _ = _
```

Learning Objectives

✓ Explain the benefits and drawbacks of data types in Haskell vs. OO classes

✓ Explain the benefits and drawbacks of type classes in Haskell

✓ Declare your own data types in Haskell

✓ Program and pattern match against custom data types in Haskell

- Write your own type classes and instances in Haskell
- Explain the difference between type synonyms (`type`) and data type declarations (`data`) in Haskell
- Use "`deriving`" clauses on data types in Haskell to derive standard type classes

Declare Your Own Type Classes in Haskell



Declaring a Type Class for Locating and Outlining a Shape

```
data Point = Point { x :: Double, y :: Double }
data Circ = Circ { centre :: Point, radius :: Double }

class Locatable a where
  locateCentre :: a → Point
  outline      :: a → (Point → Bool)

instance Locatable Point where
  locateCentre p = _
  outline p p' = _

instance Locatable Rect where
  locateCentre c = _
  outline c p = dist (centre c) p == r
```

Learning Objectives

✓ Explain the benefits and drawbacks of data types in Haskell vs. OO classes

✓ Explain the benefits and drawbacks of type classes in Haskell

✓ Declare your own data types in Haskell

✓ Program and pattern match against custom data types in Haskell

✓ Write your own type classes and instances in Haskell

- Explain the difference between type synonyms (`type`) and data type declarations (`data`) in Haskell
- Use "`deriving`" clauses on data types in Haskell to derive standard type classes



Differences Between Type Aliases and Data Declarations

Type Aliases vs. Data Declarations

```
type String = [Char]
data Bool = True | False
```

- `type` introduces an *alias.*
  - Introduces no new values or constructors. Cannot be recursive.
  - Is fully interchangeable with the type being aliased; e.g., String <~> Char.
  - Use when you want a shorter name for an existing type
- `data` defines a new data type.
  - Introduces new values and data constructors. Can be recursive.
  - Cannot be substituted for other types
  - Use when you want new values that the compiler can check.



Instance Deriving Clauses

Deriving Clauses Automatically Derive Standard Type Classes

Instead of writing this:

```
data Tree = Leaf Int | Node Tree Int Tree

instance Eq Tree where
  Leaf x     == Leaf y       = x == y
  Node l x r == Node l' y r' = l == l' && x == y && r == r'
```

Deriving Clauses Automatically Derive Standard Type Classes

We simply write this (works for `Eq`, `Ord`, `Enum`, `Read`, `Show`, and others)

```
data Tree = Leaf Int | Node Tree Int Tree deriving (Eq)
```

Learning Objectives

✓ Explain the benefits and drawbacks of data types in Haskell vs. OO classes

✓ Explain the benefits and drawbacks of type classes in Haskell

✓ Declare your own data types in Haskell

✓ Program and pattern match against custom data types in Haskell

✓ Write your own type classes and instances in Haskell

✓ Explain the difference between type synonyms (`type`) and data type declarations (`data`) in Haskell

✓ Use "`deriving`" clauses on data types in Haskell to derive standard type classes